

GPU Search

Wprowadzenie

Celem projektu było opracowanie asystenta/wyszukiwarki modeli kart graficznych. Użytkownik wpisuje zapytanie w języku naturalnym, a system dopasowuje najlepsze karty.

Przykład

Zapytanie:

„Tania karta amd radeon posiadająca 8gb vram.”

Odpowiedź:

```
{
  "brand": "AMD",
  "model": "Atari VCS 800 GPU",
  "vram_gb": 8,
  "tdp_watts": 15.0,
  "fp32_tflops": 461.2,
  "memory_bandwidth_gbs": 38.4,
  "architecture": "GCN 5.0",
  "price_per_usage_hour_cents": 207.30
}
```

Dane

Skorzystaliśmy z bazy danych kart graficznych, dostępnej pod linkiem:

<https://www.kaggle.com/datasets/ellimaaac/gpus-specs-from-1986-to-2026>

Zawiera ona szczegółowe dane techniczne kart graficznych wydawanych od roku 1986 do współczesności. Na podstawie danych w zbiorze wyliczyliśmy przykładowe ceny w dolarach za wynajem danego GPU.

Zbiór danych jest w domenie publicznej (CC0: Public Domain), więc można z niego korzystać bez ograniczeń.

Preprocessing

Aby dane nadawały się do użycia musieliśmy przekonwertować dane w różnych jednostkach pamięci (np. KB, MB/s) na jednakową skalę (GB, GB/s).

Dodatkowo wprowadziliśmy metrykę ceny za godzinę, którą oszacowaliśmy następująco:

```
def compute_price(row):
    power_cost = 0.00025 * row["tdp_watts"]
    compute_cost = 0.15 * row["fp32_tflops"]
    memory_cost = 0.000003 * row["vram_gb"]
    base = 2.0
    price = base + power_cost + compute_cost + memory_cost
    return round(price * 100, 2)
```

State-of-the-art

Współczesnym punktem odniesienia na rynku w dziedzinie ekstrakcji intencji oraz transformacji tekstu niestrukturyzowanego na sztywne obiekty danych są Duże Modele Językowe (LLM). W rozwiązaniach tej klasy nie stosuje się już ręcznie pisanych reguł gramatycznych ani lokalnych słowników odmian. Zamiast tego wykorzystuje się zaawansowany mechanizm znany jako *Function Calling* lub *Tools Integration*. Deweloper definiuje pożądany schemat wyjściowy (np. za pomocą struktur *JSON Schema*), a model LLM, dzięki głębokiemu, kontekstowemu rozumieniu semantyki,

przeprowadza automatyczną inferencję. System ten potrafi „wtopić” wyciągnięte z tekstu intencje bezpośrednio w oczekiwane klucze struktury danych, bez problemu radząc sobie z dowolnymi synonimami, slangiem technologicznym czy zawiłą fleksją językową [1].

Wdrożenie tego standardu w systemach produkcyjnych wiąże się jednak z istotnymi kompromisami inżynierskimi, które w wielu scenariuszach biznesowych dyskwalifikują to podejście na rzecz dedykowanych modeli hybrydowych. Wykorzystanie komercyjnych modeli LLM generuje stałe koszty operacyjne (opłaty za tokeny w modelu API), wprowadza zauważalny narzut opóźnień sieciowych oraz niesie ryzyko wystąpienia tzw. halucynacji (np. zwrócenia uszkodzonego lub losowego formatu JSON) [1]. Co więcej, procesowanie zapytań na zewnętrznych serwerach dostawców AI rodzi uzasadnione problemy z prywatnością danych użytkowników końcowych. Z tego powodu współczesna inżynieria oprogramowania traktuje LLM jako SOTA pod względem czystej elastyczności lingwistycznej, jednak w architekturze systemów lokalnych, bezpiecznych i zoptymalizowanych pod kątem wydajności, wciąż ustępują one miejsca w pełni przewidywalnym, deterministycznym potokom regułowo-statystycznym [2].

Metody

Wykorzystaliśmy dwa podejścia.

BERT

Dane GPU przekonwertowaliśmy na zdania w języku naturalnym:

```
def gpu_to_text(gpu):
    return (
        f"{gpu['gpu_model']} GPU. "
        f"{memory_label(gpu['vram_gb'])}, {gpu['vram_gb']} GB VRAM. "
        f"{gpu['fp32_tflops']} TFLOPS FP32 compute performance. "
        f"{gpu['memory_bandwidth']} GB/s memory bandwidth. "
        f"{price_label(gpu['price_per_hour_usd_cents'])}, "
        f"{gpu['price_per_hour_usd_cents']} cents per hour. "
        f"{gpu['architecture']} architecture."
    )
```

Następnie z użyciem modelu all-MiniLM-L6-v2 obliczaliśmy podobieństwo tych zdań z zapytaniem użytkownika:

```
model = SentenceTransformer("all-MiniLM-L6-v2")
embeddings = model.encode(sentences)
query = "64 gb vram, price in range 1500 cents per hour, from nvidia."
similarities = model.similarity(embeddings, model.encode(query))
i = int(np.argmax(similarities))
print(sentences[i])
```

Podejście to było relatywnie proste w implementacji ale dawało niezadowalające wyniki. Mimo, że dane tekstowe (np. model czy producent) często były dopasowywane poprawnie, tak dane liczbowe, z natury tego podejścia jedynie zaburzały wyniki.

Kolejną wadą tej metody jest niska wyjaśnialność wyników (brak jasnego „toku myślenia”, który doprowadził do konkretnych wyników). Dostajemy jedynie jedną liczbę świadczącą o poziomie dopasowania, ale nie dostajemy odpowiedzi, na jakiej podstawie została ona wyznaczona.

Named Entity Recognition

Niepowodzenie poprzedniego podejścia zmusiło nas do innego spojrzenia na problem. Uznaliśmy, że zamiast dopasowywać zdania w języku naturalnym, można spróbować najpierw wyciągnąć

najważniejsze dane ze zdania podanego przez użytkownika, a później dopasowywać je według prostej metryki.

Próba realizacji tego podejścia doprowadziła nas do zagadnienia *Named Entity Recognition*. Do naszego projektu użyliśmy modelu NLP dostępnego w spaCy: `pl_core_news_sm`, do którego dodaliśmy własne reguły wykrywania ważnych dla nas encji.

Przykładowa reguła wykrywania encji wygląda następująco – jest to słownik:

```
{
  "label": "VRAM",
  "pattern": [
    {"TEXT": {"REGEX": "r"^\\d+(,\\d+)?$"}},
    {"LOWER": {"IN": ["mega", "megabajt", "megabajty", "megabajtów", "giga",
      "gigabajt", "megabajtami", "gigabajty", "gigabajtów", "mb", "gb", "ram",
      "vram", "megabajta", "gigabajtami", "gigabajta"]}},
  ],
},
```

Ta konkretna reguła służy do wykrywania ilości pamięci VRAM karty, o której myśli użytkownik. Przedstawiona reguła obsługuje zapis cyfrowy, natomiast zapis słowny został zrealizowany za pomocą dodatkowych wzorców.

Mnogość różnych form słów *megabajt* i *gigabajt* na liście wynika z problemów modelu z prawidłowym lematyzowaniem tych słów (nie są to jedyne słowa, z którymi model miał kłopot). Aby poprawić skuteczność wykrywania najważniejszych dla nas tokenów, dodaliśmy reguły naprawcze do *pipeline'u* modelu, które zawierały problematyczne dla niego słowa, np.:

```
{"patterns": [[{"LOWER": "gb"}]], "attrs": {"LEMMA": "gb"}},
{"patterns": [[{"LOWER": "mb"}]], "attrs": {"LEMMA": "mb"}},
{"patterns": [[{"LOWER": "cuda"}]], "attrs": {"LEMMA": "cuda"}},
```

Jeśli model faktycznie rozpoznał dany fragment *inputu* jako interesującą nas encję, pozostawało tylko algorytmicznie wyłuskać daną liczbę, czy słowo. W wyniku działania naszego modelu, otrzymywaliśmy następujący JSON:

```
{
  "min_vram_gb": float,
  "brand": string,
  "is_cheap": bool
}
```

Ostatnim krokiem było opracowanie metryk, na podstawie których dobierano najlepszą kartę, i zastosowanie tych metryk.

Wyniki

W ramach testów, poprosiliśmy Duży Model Językowy o wygenerowanie stu przykładowych zapytań.

W ogólności drugie podejście poradziło sobie z zadaniem bardzo dobrze (z wynikiem 87/100).

Błędy w dopasowaniu wynikały głównie z problemów w rozpoznawaniu odmian niektórych słów (w szczególności *tanio* i *intel*), co wynikało z niezadowalającej jakości lematyzacji modelu (np. lemat *tanio* często stawał się *tać*).

Rozpoznawanie liczb (nawet tych pisanych słownie) działało bez zarzutów.

Wnioski

Named Entity Recognition wydaje się być dobrą metodą rozwiązania problemu wyszukiwania kart graficznych językiem naturalnym, zwłaszcza gdy zależy nam na małych (w porównaniu do wykorzystania LLM-ów) kosztach.

Aby poprawić jakość dopasowań należałoby znaleźć lepszy model NLP, lub wytrenować taki samemu jeśli mamy dostęp do znacznej ilości danych dziedzinowych.

Bibliografia

[1] OpenAI. (2023). „Function calling and other API updates.” OpenAI Technical Blog. Dostępne online: <https://openai.com/blog/function-calling-and-other-api-updates>

[2] Explosion. (2026). „Rule-based NER and Pipelines.” spaCy Usage Documentation. Dostępne online: <https://spacy.io/usage/rule-based-matching>